SESSION 04 / PRINTABLE GUIDE

Lesson 2: Land and clean nested order JSON

Session 04 | Worked example 1
Output: Lakehouse Bronze evidence, accepted/quarantine Silver tables, and a flat transfer table for Warehouse Gold.

Micro-fixture track: This four-document example explains every row. Use the main lab for 5,000 richer orders, optional structures, exploration, and scenario Gold. The tracks have separate schemas, batch/table names, and publication scripts.

What this example teaches

An application exports nested order documents. Customer attributes sit inside a structure; lines sit inside an array. This is a useful notebook lesson because it combines structural transformation, reusable acceptance rules, raw evidence, and a relational handoff.

We are not claiming SQL cannot parse JSON. We are demonstrating a notebook-based file-processing contract that becomes valuable as payload shapes and parsing requirements grow.

Prepare an isolated environment

If using this optional micro track, complete its data/target setup **before the delivery day** as part of the prerequisite gate.

Create or use an approved training Lakehouse and attach it as the notebook's default Lakehouse. Use a separate Lakehouse per participant/pair. Check the actual runtime and table/schema naming behavior before class.

Upload orders.jsonl into `Files/session_04/orders/batch-001/orders.jsonl`. It contains one complete JSON object per line, not a single multiline JSON array.

The upload supplies a local teaching source. The notebook then lands Bronze Delta evidence. This does not demonstrate a live application/API connection. An actual API implementation would additionally need approved authentication, pagination, retries, rate-limit handling, source completeness, and raw-response retention.

Create a PySpark notebook and run the following Python blocks as separate cells, in order, from a clean session. They use built-in Spark functions only. `spark` is supplied by the Fabric Spark session.

The fixed `s04_` output names below are overwritten on rerun in the isolated training Lakehouse. Raw batch files remain the retained input; these current-batch tables are not an all-batch history.

Cell 1: Declare the input contract

PYTHON

```python
from decimal import Decimal
from pyspark.sql import functions as F, types as T, Window

BATCH_ID = "s04-orders-001"
INPUT = "Files/session_04/orders/batch-001/orders.jsonl"
BRONZE = "s04_orders_bronze"
SILVER = "s04_order_lines_silver"
REJECTED = "s04_order_lines_rejected"
TRANSFER = "s04_order_lines_transfer"

spark.conf.set("spark.sql.session.timeZone", "UTC")

payload_schema = T.StructType([
    T.StructField("order_id", T.StringType()),
    T.StructField("customer", T.StructType([
        T.StructField("id", T.StringType()),
        T.StructField("name", T.StringType()),
    ])),
    T.StructField("lines", T.ArrayType(T.StructType([
        T.StructField("line_number", T.StringType()),
        T.StructField("sku", T.StringType()),
        T.StructField("quantity", T.StringType()),
        T.StructField("unit_price", T.StringType()),
    ]))),
])
```

Ask: Why keep quantities/prices as strings at the parsing boundary? Because an invalid business value must remain inspectable rather than being silently lost during type inference.

Cell 2: Persist Bronze before cleaning

```python
raw = (
    spark.read.text(INPUT)
    .select(
        F.col("value").alias("raw_json"),
        F.input_file_name().alias("source_file"),
    )
    .withColumn("batch_id", F.lit(BATCH_ID))
    .withColumn("ingested_at_utc", F.current_timestamp())
)

raw.write.format("delta").mode("overwrite").saveAsTable(BRONZE)
bronze = spark.table(BRONZE)

if bronze.count() != 4:
    raise ValueError("This fixture must contain four raw order documents.")
```

The notebook is the processing surface; `BRONZE` is a persisted Lakehouse table. A DataFrame variable or displayed result is not durable landing. The four-row assertion is specific to the supplied fixture, not a generic production threshold.

Cell 3: Expand structures while retaining evidence

```python
def normalized_key(column):
    value = F.upper(F.trim(F.col(column)))
    return F.when(F.length(value) > 0, value)


parsed = bronze.withColumn(
    "payload", F.from_json("raw_json", payload_schema)
)

flat = (
    parsed.select(
        "batch_id", "source_file", "raw_json", "ingested_at_utc",
        normalized_key("payload.order_id").alias("order_id"),
        normalized_key("payload.customer.id").alias("customer_id"),
        F.trim("payload.customer.name").alias("customer_name"),
        F.explode_outer("payload.lines").alias("item"),
    )
    .withColumn("line_number", F.expr("try_cast(item.line_number as int)"))
    .withColumn("sku", normalized_key("item.sku"))
    .withColumn("quantity_text", F.trim("item.quantity"))
```

```
    .withColumn("unit_price_text", F.trim("item.unit_price"))
    .withColumn("quantity", F.expr("try_cast(quantity_text as int)"))
    .withColumn(
        "unit_price",
        F.expr("try_cast(unit_price_text as decimal(18,2))"),
    )
)
```

`explode_outer` emits an evidence row for an empty/missing line array rather than silently losing that order. There are six actual lines plus one empty-order marker, so the expanded evidence has seven rows. That marker is not a real order line.

Raw JSON remains alongside the interpreted fields. Malformed or missing required content must fail acceptance; a production parser should additionally capture detailed parse/schema errors and version its payload contract.

Cell 4: Make the acceptance policy explicit

PYTHON

```python
line_key = Window.partitionBy("batch_id", "order_id", "line_number")

evaluated = (
    flat.withColumn("line_key_count", F.count("*").over(line_key))
    .withColumn(
        "quality_issue",
        F.when(F.col("order_id").isNull(), "MISSING_ORDER_ID")
        .when(
            F.col("line_number").isNull() | (F.col("line_number") <= 0),
            "NO_VALID_LINE",
        )
        .when(F.col("line_key_count") > 1, "DUPLICATE_LINE")
        .when(
            F.col("customer_id").isNull()
            | F.col("customer_name").isNull()
            | (F.length("customer_name") == 0),
            "MISSING_CUSTOMER",
        )
        .when(F.col("sku").isNull(), "MISSING_SKU")
        .when(
            F.col("quantity").isNull()
            | ~F.col("quantity_text").rlike(r"^[1-9][0-9]{0,5}$"),
            "INVALID_QUANTITY",
        )
        .when(
            F.col("unit_price").isNull()
            | ~F.col("unit_price_text").rlike(r"^[0-9]{1,8}(\.[0-9]{1,2})?$"),
            "INVALID_PRICE",
```

```
        ),
    )
    .withColumn(
        "line_amount",
        (F.col("quantity") * F.col("unit_price")).cast("decimal(18,2)"),
    )
)

accepted = evaluated.filter(F.col("quality_issue").isNull())
rejected = evaluated.filter(F.col("quality_issue").isNotNull())

counts = (evaluated.count(), accepted.count(), rejected.count())
if counts != (7, 3, 4):
    raise ValueError(f"Unexpected expanded/accepted/quarantine counts: {counts}")

accepted_total = accepted.agg(F.sum("line_amount")).first()[0]
if accepted_total != Decimal("296.00"):
    raise ValueError(f"Unexpected accepted-line total: {accepted_total}")
```

The teaching rules accept positive integer quantities up to six digits and nonnegative prices with up to eight integer/two fractional digits. They do not infer currencies, convert locales, remove currency symbols, or round arbitrary precision into valid business input.

Both O1003 occurrences are quarantined. `dropDuplicates()` would conceal the ambiguity without an approved winner rule. An invalid quantity is not zero.

Cell 5: Persist evidence and the Warehouse transfer contract

PYTHON

```
silver_lines = (
    accepted.select(
        "batch_id", "order_id", "line_number",
        "customer_id", "customer_name", "sku",
        "quantity", "unit_price", "line_amount",
        "source_file", "ingested_at_utc",
    )
    .withColumn("quality_rule_version", F.lit("s04-orders-v1"))
)
(
    silver_lines.write.format("delta").mode("overwrite")
    .option("overwriteSchema", "true").saveAsTable(SILVER)
)
(
    rejected.write.format("delta").mode("overwrite")
    .option("overwriteSchema", "true").saveAsTable(REJECTED)
)
```

```
transfer = spark.table(SILVER).select(
    "batch_id", "order_id", "line_number",
    "customer_id", "customer_name", "sku",
    "quantity", "unit_price", "line_amount",
)
transfer.write.format("delta").mode("overwrite").saveAsTable(TRANSFER)

spark.table(TRANSFER).orderBy("order_id", "line_number").show(truncate=False)
spark.table(REJECTED).select(
    "order_id", "line_number", "quality_issue", "raw_json"
).show(truncate=False)
```

These are separate table writes, not one transaction. Allow downstream transfer only after the whole notebook succeeds and the operator verifies the batch, counts, quarantine policy, and immutable input. Do not copy a transfer table left by a failed prior run.

Silver uses an explicit column contract, not every column left in the notebook's working DataFrame. Keep normalized business fields and useful lineage; leave raw JSON, nested `item`, temporary text conversions, and duplicate-count helpers in Bronze/quarantine or working state. The Warehouse transfer deliberately narrows Silver to nine business/batch columns. See the column-selection lesson for the reasoning.

The explicit schema-overwrite option permits this revised teaching contract to replace an earlier wide schema in the isolated lab. It is not a production schema-evolution policy and must not be used against an unknown/shared table.

Expected outcome

Order	Line	SKU	Quantity	Unit price	Accepted line amount
O1001	1	PUMP-01	2	125.50	251.00
O1001	2	FILTER-01	1	30.00	30.00
O1002	2	SEAL-01	3	5.00	15.00

Quarantine contains one `INVALID_QUANTITY`, two `DUPLICATE_LINE` rows, and one `NO_VALID_LINE` marker. The accepted-line total is **296.00**, not a claim about complete order revenue.

For this exercise, the steward explicitly approves publishing the accepted lines while retaining these known rejected rows. If the required business contract is complete orders, this policy is inappropriate: quarantine may need to block the order or the entire batch. Unexpected rejections block the teaching handoff until reviewed.

Demonstration and learner prompts

Have learners predict the expansion count before running Cell 3. Ask why an empty order still has evidence and why identical duplicate lines remain ambiguous. Then inspect a rejected row next to its raw JSON.

Rerun the same unchanged input with no competing writer. The accepted business rows and amounts should remain the same; ingestion timestamps may change. The example uses current-batch replacement and does not implement incremental orders, updates/deletes, an all-batch audit log, or production schema-evolution handling.

Next: Device telemetry, or proceed directly to Warehouse Gold publication.

Source: 02_nested_json_orders.md. Companion-material links open the repository originals.